ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Mark the logic, not small syntax errors. The ticks show where marks are given; accept other correct solutions.

SECTION A

QUESTION 1

1.1 1.1.1 D ✓ 1.1.2 B ✓ 1.1.3 C ✓ 1.1.4 A ✓ 1.1.5 B ✓ 1.1.6 D ✓ 1.1.7 A ✓ 1.1.8 C ✓ 1.1.9 C ✓ 1.1.10 A ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 True ✓ 1.2.4 False ✓ 1.2.5 True ✓ 1.2.6 True ✓ 1.2.7 True ✓ 1.2.8 True ✓ 1.2.9 True ✓ 1.2.10 False ✓ (10)

1.3 1.3.1 H ✓ 1.3.2 C ✓ 1.3.3 E ✓ 1.3.4 G ✓ 1.3.5 J ✓ 1.3.6 F ✓ 1.3.7 B ✓ 1.3.8 A ✓ 1.3.9 D ✓ 1.3.10 I ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1.1 Example:

```
const int sensorPins[5] = {A0, A1, A2, A3, A4};   ✓✓
int values[5];                                     ✓
for (int i = 0; i < 5; i++) {                      ✓
  values[i] = analogRead(sensorPins[i]);           ✓
}
```

(5)

2.1.2 Example:

```
int best = 0;                          ✓
for (int i = 1; i < 5; i++) {          ✓
  if (values[i] > values[best]) {      ✓
    best = i;                          ✓
  }
}
```

(4)

2.2 struct Motor { int pwmPin; int dirPin; int speed; }; ✓✓ Motor left = {5, 4, 0}; ✓ Motor right = {6, 7, 0}; ✓ (4)

2.3 Example:

```
unsigned long previousMillis = 0;                ✓
const long interval = 250;
bool ledState = LOW;
void loop() {
  unsigned long now = millis();                  ✓
  if (now - previousMillis >= interval) {        ✓✓
    previousMillis = now;                        ✓
    ledState = !ledState;
    digitalWrite(13, ledState);                  ✓
  }
  // other tasks continue here
}
```

(6)

[19]

QUESTION 3

3.1.1 Three states drawn ✓✓; WAIT → MEASURE when an object is detected ✓; MEASURE → SORT after 1 s ✓; SORT → WAIT when the servo has returned to 90° ✓ (5)

3.1.2 Example:

```
enum State {WAIT, MEASURE, SORT};                ✓
State state = WAIT;
unsigned long t0;
char colour;
void loop() {
  switch (state) {                               ✓
    case WAIT:
      if (objectDetected()) {                    ✓
        t0 = millis(); state = MEASURE;          ✓
      }
      break;
    case MEASURE:
      colour = readColour();                     ✓
      if (millis() - t0 >= 1000) state = SORT;   ✓
      break;
    case SORT:
      if (colour == 'R') sorter.write(30);       ✓
      else sorter.write(150);                    ✓
      delay(500);  sorter.write(90);  delay(500);  ✓
      state = WAIT;                              ✓
      break;
  }
}
```

(10)

3.2.1 Example:

```
volatile long pulses = 0;                                          ✓
void countPulse() {                                                ✓
  pulses++;                                                        ✓
}
void setup() {
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(2), countPulse, RISING);  ✓✓
}
```

(5)

3.2.2 $\text{Revolutions} = 140 \div 20 = 7$ ✓; $\text{circumference} = \pi \times 65 = 204{,}20 \text{ mm}$ ✓; $\text{distance} = 7 \times 204{,}20 = 1\,429{,}42 \text{ mm} \approx 1{,}43 \text{ m}$ ✓ (3)

[23]

QUESTION 4

4.1 Example:

```
void setMotor(int speed) {                    ✓
  speed = constrain(speed, -255, 255);        ✓
  if (speed >= 0) digitalWrite(7, HIGH);       ✓
  else digitalWrite(7, LOW);                  ✓
  analogWrite(6, abs(speed));                 ✓✓
}
```

(6)

4.2.1 $\text{left} = \text{base} + K_p \times \text{error}$ ✓; $\text{right} = \text{base} - K_p \times \text{error}$ ✓ (accept the opposite signs if explained) (2)

4.2.2 $\text{Correction} = 40 \times 1{,}5 = 60$; $\text{left} = 150 + 60 = 210$ ✓; $\text{right} = 150 - 60 = 90$ ✓ (2)

4.2.3 It reacts to how fast the error is changing ✓, which reduces overshoot / wobbling and gives smoother line following ✓. (2)

4.3 K_p: faster response, but too much causes overshoot and oscillation ✓✓; K_i: removes the steady-state error, but too much causes slow oscillation / overshoot (wind-up) ✓✓; K_d: damps the motion and reduces overshoot, but too much makes it sensitive to noise / jittery ✓✓ (6)

[18]

QUESTION 5

5.1 Example:

```
void loop() {
  if (Serial.available() > 0) {          ✓
    char cmd = Serial.read();            ✓
    switch (cmd) {                       ✓
      case 'F': forward(); break;        ✓ (all five cases)
      case 'B': back(); break;
      case 'L': left(); break;
      case 'R': right(); break;
      case 'S': stopAll(); break;        ✓
      default: break;                    ✓ (ignore unknown characters)
    }
  }
}
```

(6)

5.2 Any FOUR: print variable values to the Serial Monitor ✓; test each module (sensor, motor) separately ✓; use LEDs to show which part of the program is running ✓; use a multimeter / oscilloscope to check signals ✓; comment out sections to find the fault; check wiring and power supply; use a logic analyser for communication; read compiler error messages carefully (4)

[10]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE